text-tunnel 작업 내역

> 작성 2026-10-20 · 개발 기간 2026-10-14 ~ 2026-10-20 · 저장소 생성 2026-10-20
> 서비스 https://text-tunnel.onrender.com · 저장소 https://github.com/esstional-tool/text-tunnel (Private)
> 화면 시안 https://claude.ai/artifact/EfTwzgNi6wkzPLeLLXgiAJ
> 기능 상세(3장)와 겪은 문제(6장) 아래의 📄 **실제 코드** 를 누르면 구현·수정한 코드가 펼쳐진다 (2026-10-20 시점 사본).

내 PC와 내 휴대폰을 한 번 짝지어 두고 텍스트·사진·동영상·파일을 주고받는 개인용 메신저.
DB도 외부 저장소도 쓰지 않고, 기록을 남기지 않는 것이 처음부터 끝까지 지킨 원칙이다.

---

1. 요구사항과 결정

요구	결정	이유
DB 없이, Supabase 같은 외부 서비스도 없이	Node 서버 하나가 메모리로만 중계	운영 부담 없고 기록이 남지 않음
휴대폰에서 어디서나 접속	**Render 무료** 웹 서비스	서버가 계속 떠 있어야 실시간 중계 가능
(검토 후 기각) Vercel	서버리스는 요청마다 다른 인스턴스·메모리가 공유 안 됨 → 메시지가 상대에게 안 감	
(검토 후 기각) 오라클 무료 VM	설정이 번거로움	
(검토 후 기각) WebRTC 직접 연결	휴대폰 데이터·회사망에서 연결 실패 가능, 무료 중계(TURN) 없음	
매번 방 만들고 비밀번호 주고받기 귀찮음	**처음 한 번 QR 짝 맺기 → 앱만 열면 자동 연결**	
내 PC와 휴대폰 둘만	한 연결에 **최대 2대**	
기록 안 남기기	서버 메모리만, 파일 10분 뒤 삭제, 둘 다 나가면 즉시 삭제, 로그에 아무것도 안 씀	
브라우저 방문 기록도 지우고 싶음	크롬·엣지 **확장 프로그램**	웹사이트는 보안상 방문 기록을 못 지움
급할 때 한 번에 정리	**단축키 완전 정리** (확장)	

요구	결정	이유
(보류) PC→PC 원격지원	2026-10-14 보류. "핵심은 메신저 전송"	
(보류) PC 실행파일(.exe)	작업 표시줄 아이콘을 숨기려면 필요하지만 번거로워서 현재 방식 유지	

---

2. 전체 구조

```
[PC 브라우저/설치앱]                  [Render: node server.js]                [휴대폰 브라우저/설치앱]
  public/ 화면   ── HTTPS ──>  정적 파일 + /api/*  <── HTTPS ──  public/ 화면
        └ 실시간 수신: SSE(/api/events) ┘        └ 메모리: rooms / tokens / pairs / files
[엣지 확장 extension/]  ← postMessage →  화면     (디스크·로그에 아무것도 안 씀)
```

- **서버** `server.js` — 의존성 0 (Node 기본 모듈만). 정적 파일, 짝 맺기, 방 입장, 실시간 전달(SSE), 조각 업로드, 파일 내려주기, 대화 지우기, 확장 zip 생성.
- **화면** `public/` — 순수 HTML·CSS·JS PWA. 휴대폰·PC 같은 화면(폭에 맞춰 늘고 줄어듦), 다크 모드 자동.
- **확장** `extension/` — Manifest V3. 방문 기록·관련 검색 기록·다운로드 목록·캐시 삭제, 단축키 완전 정리.
- **라이브러리** `public/vendor/qrcode.js` — QR 생성 (qrcode-generator 1.4.4, MIT). 외부 CDN 없이 같이 배포.

파일

파일	내용
`server.js`	서버 전부 (약 400줄)
`public/index.html`	화면 구성 (깨우기·처음·코드 입력·QR·설치 안내·대화·방문 기록 지우기·메뉴·확인창·사진 보기)
`public/app.js`	화면 동작 전부
`public/style.css`	디자인 (라이트/다크 토큰)
`public/sw.js`	PWA 서비스워커 (캐시 안 함, 안드로이드 공유 메뉴 수신)
`public/manifest.json`	PWA 설정, 아이콘, 공유 대상(share_target)
`public/icon*.png`, `icon.svg`	앱 아이콘

파일	내용
`extension/*`	크롬·엣지 확장 (현재 1.2.0)
`start.cmd`	Windows 에서 로컬 서버 실행
`package.json`	Render 배포용 (`node server.js`)

---

3. 기능 상세

3.1 짝 맺기와 연결

1. PC에서 **QR 띄우기** → 서버가 32바이트 비밀 열쇠와 **6자리 코드**(5분·1회용, 헷갈리는 0/O/1/I/L 제외)를 만든다.
2. 휴대폰 기본 카메라로 QR을 찍으면 `/#pair=코드` 로 열리고, 코드를 내면 같은 열쇠를 받는다. QR이 안 되면 코드 직접 입력.
3. 두 기기는 열쇠를 각자 `localStorage` 에 보관. 앱을 열면 열쇠로 `/api/join` → 서버는 **열쇠의 SHA-256 을 방 번호**로 쓴다(열쇠 자체는 보관 안 함).
4. 방은 최대 2대. 같은 기기(기기 ID 동일)가 다시 들어오면 자리를 이어받는다.
5. 상대를 기다리는 동안 **대기 화면에 QR·코드를 상시 표시**(5분마다 갱신) → 열쇠를 잃은 기기도 다시 찍기만 하면 됨.
6. 짝 맺은 휴대폰에 **앱 설치 안내**: 안드로이드는 설치 버튼, 아이폰은 공유 → 홈 화면에 추가 순서.
   아이폰은 사파리와 홈 화면 앱의 저장소가 달라서 설치한 앱을 처음 열 때 코드를 한 번 입력.

실제 코드

▼ 실제 코드 — `server.js` 213–235 · 짝 코드 발급·사용

GitHub에서 보기

```
  // 짝 맺기 시작: 열쇠를 가진 기기가 6자리 코드를 받는다. { key? } — 없으면 새 열쇠를 만든다
  if (p === '/api/pair/start' && req.method === 'POST') {
    return readJson(req, (j) => {
      const key = typeof j?.key === 'string' && /^[A-Za-z0-9_-]{43}$/.test(j.key) ? j.key :
crypto.randomBytes(32).toString('base64url');
      const code = newCode();
      pairs.set(code, { key, timer: setTimeout(() => pairs.delete(code), PAIR_TTL) });
      send(res, 200, { code, key, expiresIn: PAIR_TTL / 1000 });
    });
  }

  // 짝 맺기 완료: 다른 기기가 코드를 내고 열쇠를 받는다. 코드는 한 번 쓰면 사라진다
```

```
  if (p === '/api/pair/claim' && req.method === 'POST') {
    const ip = clientIp(req);
    if (blocked(ip)) return send(res, 429, { error: '코드를 너무 많이 틀렸어요. 10분 뒤에 다시 해 주세요' });
    return readJson(req, (j) => {
      const code = String(j?.code || '').toUpperCase().replace(/[^A-Z0-9]/g, '');
      const pair = pairs.get(code);
      if (!pair) { miss(ip); return send(res, 404, { error: '코드가 맞지 않거나 시간이 지났어요. PC에서 새 코드를 받아 주세요' }); }
      clearTimeout(pair.timer);
      pairs.delete(code);
      send(res, 200, { key: pair.key });
    });
  }
```

▼ 실제 코드 — `public/app.js` 80–90 · 대기 화면 QR 자동 갱신

GitHub에서 보기

```
async function refreshWaitQr() {
  clearTimeout(waitTimer); waitTimer = null;
  if (DEMO) { renderQr($('waitQr'), 'K7M4QX'); $('waitCode').textContent = fmtCode('K7M4QX'); return; }
  if (others || !key || document.hidden || $('chat').hidden) return;
  const r = await post('pair/start', { key }).catch(() => null);
  if (!r?.ok || others) return;
  const j = await r.json();
  renderQr($('waitQr'), j.code);
  $('waitCode').textContent = fmtCode(j.code);
  waitTimer = setTimeout(refreshWaitQr, (j.expiresIn - 15) * 1000);
}
```

3.2 연결 유지

- 실시간 수신은 **SSE**(서버 → 브라우저 한 방향 스트림), 보내기는 일반 POST.
- 연결이 끊기면(화면 꺼짐·새로고침) **30초 유예** 후 자리 정리.
- 앱이 **20초마다 살아 있음 신호**. 앱을 강제 종료해 서버가 모르는 **유령 자리**는, 방이 꽉 찼을 때 신호가 90초 넘게 없는 자리부터 비운다. 막힌 쪽은 15초 뒤 자동 재시도.
- 창·앱을 닫으면 `sendBeacon` 으로 바로 나감 → 둘 다 닫으면 방이 즉시 사라짐.
- **놓친 메시지 다시 받기**(이슈 #9): 메시지·파일·대화 지우기 이벤트에 `id: 방시작표시:순번` 을 붙이고 방마다 최근 2분(최대 300개)을 메모리에 보관. 다시 연결하면 마지막으로 받은 번호 이후만 다시 보냄(SSE Last-

Event-ID / `?last=`). 화면은 메시지 ID 로 중복을 거르고, 기기 표식(기기 ID 해시)으로 이전 접속에서 보낸 내 메시지를 알아봄. 대화 지우기 시 보관분도 비움.

- Render 무료 서버가 잠들어 있으면 "**서버 깨우는 중**" 화면 (최대 1분).

실제 코드

▼ 실제 코드 — `server.js` 276–292 · SSE 연결 유지·30초 유예

GitHub에서 보기

```
  if (p === '/api/events' && req.method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'text/event-stream', 'Cache-Control': 'no-store',
Connection: 'keep-alive', 'X-Accel-Buffering': 'no' });
    res.write('retry: 2000\n\n');
    clearTimeout(member.timer);
    member.res?.end(); // 같은 탭이 재접속하면 이전 연결을 정리
    member.res = res;
    presence(room);
    const ping = setInterval(() => res.write(': ping\n\n'), 25000);
    req.on('close', () => {
      clearInterval(ping);
      if (member.res !== res) return;
      member.res = null;
      presence(room);
      member.timer = setTimeout(() => removeMember(room, token), GRACE);
    });
    return;
  }
```

▼ 실제 코드 — `public/app.js` 799–803 · 20초 살아 있음 신호

GitHub에서 보기

```
setInterval(async () => {
  if (DEMO || !token || document.hidden) return;
  const r = await fetch(api('ping')).catch(() => null);
  if (r?.status === 401) { es?.close(); token = null; joinRoom(); }
}, 20000);
```

3.3 메시지·사진·파일

- **텍스트**: 말풍선 탭하면 복사, 링크 자동 연결, 같은 사람·같은 분은 카톡식으로 묶음.
- **+ 첨부 패널**: 앨범(사진·동영상 선택 화면) / 카메라(휴대폰) / 파일.

- **보내기 전 미리보기**: 썸네일 확인·빼기, **일반 화질**(긴 변 2048px JPEG, 1.5MB 이상만) / **원본 화질**.
- PC: **끌어다 놓기·Ctrl+V 붙여 넣기**도 미리보기로.
- 업로드는 **8MB 조각**으로 순서대로 (프록시 요청 크기 제한 회피). 한 파일 최대 100MB(Render), 서버 전체 300MB.
- 한 번에 보낸 사진은 **앨범 말풍선**(바둑판) + 모두 저장. 사진을 누르면 **크게 보기**(스와이프·화살표 키·저장).
- 동영상은 iOS 재생을 위해 Range(부분) 응답 지원.
- 안드로이드 **공유 메뉴** → **Tunnel** 로 보내기 (설치한 앱에서).
- 다른 탭을 보고 있으면 탭 제목에 안 읽은 개수.

실제 코드

▼ 실제 코드 — `public/app.js` 434–487 · 보내기 전 미리보기

GitHub에서 보기

```
/* ── 보내기 전 미리보기 (트레이) ──────────── */
let tray = []; // { file, url }
function addToTray(files) {
  const list = [...files];
  if (list.some((f) => !f.size)) toast('빈 파일은 뺐어요');
  for (const file of list.filter((f) => f.size)) {
    tray.push({ file, url: kindOf(file) === 'file' ? null : URL.createObjectURL(file) });
  }
  closeAttach();
  renderTray();
}

function renderTray() {
  $('tray').hidden = !tray.length;
  const list = $('trayList'); list.innerHTML = '';
  tray.forEach((t, i) => {
    const th = el('div', 'thumb');
    const k = kindOf(t.file);
    if (k === 'image') { const img = new Image(); img.src = t.url; img.alt = t.file.name;
th.append(img); }
    else if (k === 'video') {
      const v = el('video'); v.src = t.url + '#t=0.1'; v.muted = true; v.preload =
'metadata'; v.playsInline = true;
      th.append(v, el('span', 'badge', '동영상'));
    } else {
      const d = el('div', 'doc-thumb'); d.append(svg(ICON_FILE), el('span', null,
t.file.name)); th.append(d);
    }
    const x = el('button', 'remove'); x.type = 'button'; x.setAttribute('aria-label',
`${t.file.name} 빼기`); x.append(svg('M6 6l12 12M18 6L6 18'));
    x.onclick = () => { if (t.url) URL.revokeObjectURL(t.url); tray.splice(i, 1);
```

```
renderTray(); };
    th.append(x);
    list.append(th);
  });
  const imgs = tray.filter((t) => kindOf(t.file) === 'image').length;
  const vids = tray.filter((t) => kindOf(t.file) === 'video').length;
  const docs = tray.length - imgs - vids;
  $('originalWrap').hidden = !imgs;
  $('trayInfo').textContent = [imgs && `사진 ${imgs}`, vids && `동영상 ${vids}`, docs && `파일 ${docs}`].filter(Boolean).join(' · ')
    + ` · ${fmtSize(tray.reduce((s, t) => s + t.file.size, 0))}`;
  $('traySend').textContent = `${tray.length}개 보내기`;
  syncInput();
  list.scrollLeft = list.scrollWidth; // 방금 고른 것이 보이게 (scrollIntoView 는 바깥 페이지까지 움직여서 쓰지 않음)
}

function clearTray() {
  for (const t of tray) if (t.url) URL.revokeObjectURL(t.url);
  tray = []; renderTray();
}

function sendTray() {
  if (!tray.length) return;
  const files = tray.map((t) => t.file);
  const original = $('original').checked;
  clearTray();
  sendFiles(files, original);
}
```

▼ 실제 코드 — `public/app.js` 269–293 · 앨범 말풍선

GitHub에서 보기

```
// 사진은 앨범 말풍선에 담는다. 한 번에 고른 사진(같은 batch)은 같은 말풍선에 이어 붙인다
function addToAlbum(album, src, m) {
  const img = new Image(); img.src = src; img.alt = m.fname; img.loading = 'lazy';
  img.dataset.fname = m.fname;
  img.onclick = () => openViewer(img);
  const foot0 = album.querySelector('.album-foot');
  if (foot0) foot0.before(img); else album.append(img);
  const n = album.querySelectorAll('img').length;
  album.className = `bubble album ${n === 1 ? 'n1' : n === 2 ? 'n2' : n === 3 ? 'n3' : n === 4 ? 'n4' : 'nmany'}`;
  if (n >= 2) {
    let foot = album.querySelector('.album-foot');
```

```
    if (!foot) {
      foot = el('div', 'album-foot'); foot.append(el('span'), el('button', null, '모두 저
장'));
      foot.querySelector('button').onclick = () => saveAll(album);
      album.append(foot);
    }
    foot.querySelector('span').textContent = `사진 ${n}장`;
  }
}

function saveAll(album) {
  for (const img of album.querySelectorAll('img')) {
    const a = el('a'); a.href = img.src; a.download = img.dataset.fname;
document.body.append(a); a.click(); a.remove();
  }
}
```

▼ 실제 코드 — `public/app.js` 399–432 · 묶음 보내기

GitHub에서 보기

```
async function sendFiles(files, original) {
  files = files.filter((f) => f.size > 0);
  if (!files.length) return;
  const batch = randId().replace(/[^\w-]/g, '').slice(0, 40);
  if (DEMO) {
    for (const f of files) add({ type: 'file', from: myId, fname: f.name, mime: f.type,
size: f.size, batch, at: Date.now(), src: URL.createObjectURL(f) });
    return;
  }
  const imgs = files.filter((f) => kindOf(f) === 'image').length;
  const what = imgs === files.length ? `사진 ${imgs}장` : `${files.length}개`;
  const li = el('li', 'row mine');
  const line = el('div', 'line');
  const b = el('div', 'bubble pending');
  const label = el('div', null, `${what} 보내는 중…`);
  const bar = el('div', 'bar'); const fill = el('i'); bar.append(fill);
  b.append(label, bar); line.append(b); li.append(line);
  place(li);

  const prepared = [];
  for (const f of files) prepared.push(original || kindOf(f) !== 'image' ? f : await
shrink(f));
  const total = prepared.reduce((s, f) => s + f.size, 0);
  let done = 0;
  const failed = [];
```

```
  for (const [i, f] of prepared.entries()) {
    if (files.length > 1) label.textContent = `${what} 보내는 중… ${i +
1}/${files.length}`;
    try {
      await uploadOne(f, kindOf(f) === 'image' ? batch : null, (sent) => { fill.style.width
= `${((done + sent) / total) * 100}%`; });
    } catch (e) { failed.push(`${f.name}: ${e.message}`); }
    done += f.size;
  }
  if (!failed.length) return li.remove();
  b.classList.add('failed'); bar.remove();
  label.textContent = `${failed.length}개를 못 보냈어요 · ${failed.join(', ')}`;
}
```

3.4 기록을 남기지 않는 장치

대상	처리
서버 메모리의 사진·파일	10분 뒤 자동 삭제, 대화 지우기·둘 다 나가기 시 즉시 삭제
서버 로그	시작 문구 외 아무것도 안 씀 (방·코드·열쇠 출력 없음)
브라우저 캐시	파일 응답 `Cache-Control: no-store`
화면의 대화	새로고침·창 닫기 시 사라짐. **헤더 휴지통**(확인 없이 즉시) / 메뉴 **대화 지우기** → 양쪽 화면 + 서버 파일
이 기기에 남는 것	연결 열쇠·기기 이름·기기 ID 뿐

실제 코드

▼ 실제 코드 — `server.js` 294–301 · 대화 지우기 (서버 파일 즉시 삭제)

GitHub에서 보기

```
  // 대화 지우기: 서버에 남은 사진·파일과 올라오던 조각을 즉시 버리고, 양쪽 화면도 비우게 한다
  if (p === '/api/clear' && req.method === 'POST') {
    for (const f of room.files.values()) { clearTimeout(f.timer); fileBytes -=
f.buf.length; }
    for (const u of room.uploads.values()) clearTimeout(u.timer);
    room.files.clear(); room.uploads.clear();
    broadcast(room, { type: 'clear', from: member.id, name: member.name, at: Date.now() });
```

```
    return send(res, 200, 'ok');
  }
```

▼ 실제 코드 — `public/app.js` 744–749 · 헤더 휴지통

GitHub에서 보기

```
// 헤더 휴지통: 확인 없이 바로 양쪽 대화·서버 파일 삭제
$('trashBtn').onclick = () => {
  if (!$('scroller').classList.contains('has-msgs') && !tray.length) return toast('지울 대화가 없어요');
  clearTray();
  clearChat();
};
```

▼ 실제 코드 — `server.js` 139–147 · 파일 응답 no-store

GitHub에서 보기

```
function serveFile(f, req, res) {
  const total = f.buf.length;
  const head = {
    'Content-Type': f.mime,
    'Accept-Ranges': 'bytes',
    'Cache-Control': 'no-store', // 사진·동영상이 브라우저 캐시 폴더에 남지 않게
    'Content-Disposition': `inline; filename*=UTF-8''${encodeURIComponent(f.name)}`,
  };
  const m = /bytes=(\d*)-(\d*)/.exec(req.headers.range || '');
```

3.5 확장 프로그램 "Text Tunnel 흔적 지우기" (1.2.0)

- 앱 메뉴 → **방문 기록 지우기**: zip 받기(`/text-tunnel-cleaner.zip` , 서버가 시작할 때 `extension/` 을 묶음), 설치 5단계, 설치 여부·버전 표시, 새 버전 안내.
- **자동 삭제**(기본 켜짐): 앱 방문·관련 검색을 하는 즉시 기록 삭제.
- **지금 지우기**: 앱 방문 기록 + 관련 검색 기록(구글·빙·네이버·다음에서 `text tunnel` / `text-tunnel` / `onrender` 검색) + 다운로드 목록 + 캐시. 연결 열쇠는 유지.
- **완전 정리 단축키**(기본 Ctrl+Shift+9, 사용자 PC 는 **Ctrl+Shift+8**): 열린 앱에 대화 지우기·방 나가기·연결 정보 삭제 → 앱 창 닫기 → 사이트 데이터(열쇠 포함)·캐시·쿠키·방문 기록·다운로드 목록 삭제. 다시 쓰려면 QR 재연결.

- 앱 창이 앞에 있으면 **앱이 직접 단축키를 받음**(확장에 설정된 키를 따라감) — 전역 등록이 다른 프로그램과 충돌해도 동작.
- 다른 사이트 기록, 다른 검색, 다른 localhost 포트는 건드리지 않음.

실제 코드

▼ 실제 코드 — `extension/background.js` 28–68 · 지금 지우기 · 완전 정리

GitHub에서 보기

```
async function deleteHistory() {
  const seen = new Set();
  for (const text of ['text-tunnel', 'Text Tunnel', 'text tunnel', 'tunnel', 'onrender',
'8787']) {
    const items = await chrome.history.search({ text, startTime: 0, maxResults: 100000 });
    for (const it of items) if (shouldDelete(it.url)) seen.add(it.url);
  }
  for (const url of seen) await chrome.history.deleteUrl({ url });
  return seen.size;
}

// 다운로드 목록에서만 뺀다 (받은 파일은 그대로)
async function eraseDownloads() {
  const erased = await chrome.downloads.erase({ urlRegex: 'text-
tunnel\\.onrender\\.com|localhost:8787' }).catch(() => []);
  return erased.length;
}

// 방문 기록·검색 기록·다운로드 목록 + 캐시. 연결 열쇠(localStorage)는 지우지 않는다 (다시 짝
맺을 필요 없게)
async function clean() {
  const count = await deleteHistory();
  const downloads = await eraseDownloads();
  await chrome.browsingData.remove({ origins: ORIGINS }, { cache: true, cacheStorage: true
});
  await chrome.storage.local.set({ lastClean: Date.now() });
  return { count, downloads };
}

// 완전 정리: 열린 앱에 대화 지우기·연결 해제를 시키고, 창을 닫고, 사이트 데이터·캐시·기록을 모
두 지운다
async function wipe() {
  const tabs = (await chrome.tabs.query({ url: TAB_URLS })).filter((t) => isOurs(t.url));
  await Promise.all(tabs.map((t) => Promise.race([
    chrome.tabs.sendMessage(t.id, { type: 'wipe' }).catch(() => null),
    new Promise((ok) => setTimeout(ok, 1500)), // 응답이 늦어도 정리는 계속한다
  ])));
  await Promise.all(tabs.map((t) => chrome.tabs.remove(t.id).catch(() => null)));
```

```
  await chrome.browsingData.remove({ origins: ORIGINS }, {
    cache: true, cacheStorage: true, cookies: true, localStorage: true, indexedDB: true,
serviceWorkers: true, fileSystems: true,
  });
  const count = await deleteHistory();
  const downloads = await eraseDownloads();
  await chrome.storage.local.set({ lastWipe: Date.now() });
  return { count, downloads, closed: tabs.length };
}
```

---

4. 진행 이력

2026-10-14 — 기획·첫 버전

- DB 없이 동시 접속 기기끼리만 주고받는 구조로 결정, 견적 비교(P2P / 내 PC 서버 / 클라우드 저장소).
- 의존성 0 Node 서버 + PWA 첫 구현, 서버 비밀번호 하나로 입장.
- 카톡·아이메시지형 UI 시안 → 사용자 선택 "카톡형, 양방향 같은 화면".
- 큰 파일 조각 업로드, 안드로이드 공유 메뉴, 안 읽은 개수, PNG 아이콘.
- PC→PC 원격지원 검토 후 **보류**.

2026-10-20 — 배포와 기능 확장

- 방 이름 + 방 비밀번호, 다 나가면 방 폭파.
- 배포처 결정: Vercel 기각(서버리스) → Render 채택.
- GitHub CLI 설치(사용자 PATH, 영구 로그인), 조직 `esstional-tool` 에 Private 저장소 생성, 만든 날짜 표기.
- 조직 프로필 사진(공구함) 제작.
- Render 배포 (Singapore 권장, Free, `node server.js`).
- 이후 기능은 모두 **브랜치 → 커밋 → PR → "확정" 시 main 머지 → Render 자동 재배포** 흐름.

PR	머지 (UTC)	내용
#1	00:19	QR 짝 맺기, 2대 제한, 앱 설치 안내, 서버 깨우기 화면
#2	00:30	휴대폰을 껐다 켜면 못 들어가던 문제 (유령 자리 정리, 대기 화면 QR 상시 표시, 미니 브라우저 안내)

PR	머지 (UTC)	내용
#3	01:03	방문 기록 지우기 확장 1.0 + 앱 안 다운로드·설치 안내, 파일 캐시 금지
#4	01:04	카톡식 사진·동영상 보내기(첨부 패널·미리보기·화질·앨범·크게 보기), 대화 지우기
#5	01:04	확장 1.1: 단축키 완전 정리
#6	01:41	앱이 단축키 직접 받기·확장 설정 따라가기, 확장 1.2(검색 기록·다운로드 목록), 새 버전 안내, 헤더 휴지통
#7	02:27	문서: 작업 내역·아키텍처 노트·흐름 해설·공부 로드맵, 실제 코드 펼치기, 시험 스크립트
#8	02:33	문서: 인쇄용 PDF 와 생성 스크립트
#10	—	**이슈 #9** 연결이 잠깐 끊긴 동안 온 메시지가 사라지던 문제 → 최근 2분 다시 받기

---

5. 시험 기록

시험	대상	결과
방 기능 17항목 (생성·중복·비밀번호·격리·폭파·유예)	로컬, Render	통과 (당시 방식)
짝 맺기 16항목 (코드 발급·1회용·차단·2대 제한·재입장·격리·새 방)	로컬 / Render 15항목(차단 시험 제외)	통과
유령 자리 정리 6항목	로컬 (STALE_SEC=3)	통과
놓친 메시지 다시 받기 10항목 (놓친 것만·순서·중복 없음·처음 열기·내 메시지 표식·지우기 후·만료·새 방)	로컬 (REPLAY_SEC=3)	통과
화면 다시 받기 3항목 (닫혀 있던 동안 온 메시지 표시·내 말풍선·재연결 중복 없음)	실제 Edge, 원격 디버깅	통과
조각 업로드 (20MB 3조각, 원본 일치, 순서 어긋남 409, 용량 초과 413)	로컬	통과
확장 zip (Windows 압축 해제, 파일 6개 원본 일치)	로컬, Render	통과
완전 정리 (휴대폰 대화 지우기 알림·PC 방 나감·탭 닫힘·열쇠 삭제)	실제 Edge + 확장, 원격 디버깅 자동 조작, 키보드 단축키 입력 포함	통과
검색 기록 판별 14항목, 단축키 해석, 버전 비교	단위 시험	통과

시험	대상	결과
사진 묶음 전달·캐시 금지·대화 지우기·지운 뒤 410	Render	통과

사용자가 실제 기기로 확인한 것: QR 연결, 앱 재실행 시 자동 연결, 확장 설치(엣지), 단축키 Ctrl+Shift+8 완전 정리.

아직 실제로 확인 못 한 것: 휴대폰 '앨범' 버튼의 사진 선택 화면, 아이폰 전체 흐름, 실제 방문·검색 기록·다운로드 목록 삭제 결과(헤드리스는 기록을 안 남김), 안드로이드 공유 메뉴.

---

6. 겪은 문제와 해결

문제	원인	해결
로그인·대화 화면이 같이 보임	`display` 가 `hidden` 속성을 덮어씀	`[hidden]{display:none!important}`
휴대폰이 "사이트에 연결할 수 없음"	로그의 내부 주소(10.x, localhost:10000)로 접속	안내 + Render 에서 내부 주소 출력 안 함
휴대폰 재접속 불가	카메라·렌즈 미니 브라우저에 열쇠 저장 / 강제 종료로 유령 자리	대기 화면 QR 상시 표시, 살아 있음 신호·유령 정리
확장 설치했는데 "설치되지 않음"	확장은 크롬에, 앱은 엣지로 설치돼 있었음	엣지에 설치 (안내에 브라우저별 주소 표시)
Ctrl+Shift+9 무반응	전역 단축키는 Windows 에서 한 프로그램만 등록 가능 → 다른 프로그램이 선점	Ctrl+Shift+8 로 변경, 앱이 직접 키 받기
PR #4 가 자동으로 닫힘	아래 PR(#3) 머지 시 브랜치 삭제 → 그 브랜치를 기준으로 한 PR 닫힘	브랜치 복구·reopen·기준을 main 으로. 이후 쌓은 PR 은 먼저 기준을 바꾸고 머지
시안 페이지가 엉뚱하게 스크롤	iframe 안 `scrollIntoView` 가 바깥 페이지까지 움직임	목록 안에서만 `scrollLeft` 조정

실제 코드

▼ 실제 코드 — `public/style.css` 36 · 화면 겹침 해결: [hidden]

GitHub에서 보기

```
[hidden] { display: none !important; }
```

▼ 실제 코드 — `server.js` 370–375 · Render 에선 내부 주소 안내 생략

GitHub에서 보기

```
server.listen(PORT, () => {
  console.log(`text-tunnel 실행 중: http://localhost:${PORT}`);
  if (process.env.RENDER) return; // Render 안에서는 내부 주소라 의미 없음
  for (const nets of Object.values(require('os').networkInterfaces()))
    for (const n of nets || []) if (n.family === 'IPv4' && !n.internal) console.log(`같은
와이파이에서: http://${n.address}:${PORT}`);
});
```

▼ 실제 코드 — `server.js` 252–259 · 유령 자리 정리

GitHub에서 보기

```
      // 같은 기기가 다시 들어오면(새로고침·앱 재실행) 이전 자리를 비운다
      for (const [t, m] of room.members) if (m.device === device) dropMember(room, t);
      // 꽉 찼으면 소식이 끊긴 자리(앱 강제 종료 등으로 남은 유령)를 가장 오래된 것부터 비운다
      if (room.members.size >= ROOM_SIZE) {
        const stale = [...room.members].filter(([, m]) => Date.now() - m.seen >
STALE).sort(([, a], [, b]) => a.seen - b.seen);
        if (stale.length) { dropMember(room, stale[0][0]); presence(room); }
      }
      if (room.members.size >= ROOM_SIZE) return send(res, 409, { error: '이미 두 기기가 연
결돼 있어요' });
```

▼ 실제 코드 — `public/app.js` 617–622 · 브라우저별 확장 주소 안내

GitHub에서 보기

```
function openExt() {
  closeSheets();
  show('extScreen');
  $('extUrl').textContent = /Edg\//.test(navigator.userAgent) ? 'edge://extensions' :
'chrome://extensions';
  renderExt(); toExt({ type: 'ping' });
}
```

▼ 실제 코드 — `public/app.js` 754–770 · 앱이 직접 단축키 받기

GitHub에서 보기

```
// 앱 창이 앞에 있을 때의 완전 정리 단축키. 확장의 전역 단축키는 다른 프로그램이 같은 키를 먼저 잡으면
// Windows 가 소리 없이 무시하므로, 앱이 직접 받아 확실히 동작하게 한다.
// 확장에 설정된 단축키(예: 'Ctrl+Shift+8')를 따라가고, 확장이 없으면 Ctrl+Shift+9.
// 키보드 배열·한글 입력과 상관없게 e.code(물리 키 위치)로 비교한다
function wipeKey() {
  const parts = (ext?.shortcut || 'Ctrl+Shift+9').split('+').map((s) => s.trim());
  const last = parts.pop() || '';
  const code = /^\d$/.test(last) ? `Digit${last}` : /^[A-Z]$/i.test(last) ? `Key${last.toUpperCase()}` : last;
  return { code, ctrl: parts.includes('Ctrl'), shift: parts.includes('Shift'), alt: parts.includes('Alt') };
}
document.addEventListener('keydown', (e) => {
  const k = wipeKey();
  if (e.code !== k.code || e.ctrlKey !== k.ctrl || e.shiftKey !== k.shift || e.altKey !== k.alt) return;
  e.preventDefault();
  if (ext) return toExt({ type: 'wipe' }); // 확장이 앱 창 닫기·사이트 데이터·방문 기록까지 지운다
  wipeHere().then(() => { try { window.close(); } catch {} }); // 확장이 없으면 대화·연결 정보까지만
}, true);
```

▼ 실제 코드 — `public/app.js` 472 · scrollIntoView 대신 scrollLeft

GitHub에서 보기

```
  list.scrollLeft = list.scrollWidth; // 방금 고른 것이 보이게 (scrollIntoView 는 바깥 페이지까지 움직여서 쓰지 않음)
```

---

7. 운영 메모

- **배포**: main 에 머지하면 Render 가 자동 재배포(약 20초). 앱은 다시 열면 새 버전. **확장은 자동 업데이트 안 됨** → zip 다시 받아 기존 폴더에 덮어쓰기 → 확장 페이지 새로고침.
- **Render 무료**: 15분 미사용 시 잠듦(첫 접속 최대 1분). 깨어 있는 시간 월 750시간(워크스페이스 공유) — 서비스 하나는 24시간 켜도 한도 안. 메모리 512MB.

- **환경변수** (없어도 됨): `MAX_MB` (100), `MEM_MB` (300), `MAX_ROOMS` (50), `STALE_SEC` (90). `PORT` 는 Render 가 넣음.
- **로컬 실행**: `start.cmd` 또는 `node server.js` → http://localhost:8787 (화면만 보기: `/#demo`, `/#demo-tray` 등 장면 지정). 휴대폰은 같은 와이파이에서 PC IP:8787.
- **이 PC 도구**: Node `C:\Program Files\nodejs\node.exe` (PATH 미등록), GitHub CLI `C:\Users\KUP\AppData\Local\Programs\gh\bin\gh.exe` (사용자 PATH 등록, 자격 증명 관리자에 영구 로그인).
- **회사 PC 주의**: "조직에서 관리하는 브라우저" — 회사가 따로 수집하는 기록과 회사 네트워크의 접속 기록 (`text-tunnel.onrender.com`)은 앱·확장으로 지울 수 없다.

---

8. 앞으로 할 수 있는 것

항목	내용	비고
확장 스토어 등록	엣지 추가 기능(무료)·크롬 웹 스토어(5달러) → 클릭 한 번 설치·자동 업데이트	계정·개인정보 처리방침·심사 필요
작업 표시줄 아이콘 숨기기	지금은 `edge://apps` 에서 '창으로 열기' 끄기(탭으로 열기)로 대응	완전히 숨기려면 WebView2 기반 exe(수 MB)
아이콘·이름 눈에 안 띄게	매니페스트 이름·아이콘 교체	설치된 앱은 재설치해야 확실
PC→PC 원격지원	상주 에이전트 + 중계, 접속 중 표시 필수, 약 5일	보류 중, 회사 보안 승인 필요